Topics:

1. **random module usage**
2. **OTP Generation**
3. **Modularization in Python**
4. **Small Flask Project Example using OTP & Modularization**

1. random Module Usage (Simple Explanation)

The **random** module helps us generate random numbers or random selections.

```
import random


# Generate random integer between 1 and 10
num = random.randint(1, 10)
print(num)
```

This will generate any integer **from 1 to 10**, including **1** and **10**.

```
# Pick a random item from a list
colors = ["red", "green", "blue"]
print(random.choice(colors))


# Generate a random decimal value
print(random.random())  # Value between 0.0 to 1.0
```

This function returns a **floating value between 0.0 and 1.0**.

- **0.0 is included**
- **1.0 is *not* included**

Why we need random?

- For **OTP generation**
- For **shuffling** items
- For **games**

- For **security purposes** sometimes

---

2. OTP Generation

An OTP is usually a **4 or 6 digit random code** sent for verification.

```
import random


def generate_otp():
    otp = random.randint(1000, 9999)  # 4-digit OTP
    return otp


print(generate_otp())
```

OTP as String

```
import random
import string


def generate_otp():
    digits = string.digits  # '0123456789'
    otp = ''.join(random.choice(digits) for _ in range(6))
    return otp


print(generate_otp())
```

---

3. Modularization in Python (Simple Explanation)

Modularization means **splitting your program into multiple small files (modules)** instead of keeping everything in one file.

Why Modularization?

- Code looks **clean**
- Easy to **reuse**

- Easy to **debug**
- Encourages **teamwork**

Example Structure

project/

├── app.py # main file

├── otp_utils.py # Contains OTP functions

otp_utils.py

```
import random


def generate_otp():
   return random.randint(1000, 9999)
```

app.py

```
from otp_utils import generate_otp


print("Your OTP is:", generate_otp())
```

---

4. Mini Flask Project: OTP Verification

Project Goal

- User enters email/phone
- App generates OTP
- User verifies OTP

Project Structure

flask_otp_app/

├── app.py

├── otp_utils.py

├── templates/

  ├── send.html

  ├── verify.html

Step 1: otp_utils.py

```
import random

def generate_otp():
    return random.randint(1000, 9999)
```

Step 2: app.py

```
from flask import Flask, request, render_template, redirect, url_for, session
from otp_utils import generate_otp

app = Flask(__name__)
app.secret_key = "abc123"   # Required to store data in session

@app.route("/")
def send_page():
    return render_template("send.html")

@app.route("/send_otp", methods=["POST"])
def send_otp():
    phone = request.form['phone']
    otp = generate_otp()
    session['otp'] = otp  # storing OTP in session
    print("Generated OTP (for testing):", otp)
    return redirect(url_for("verify_page"))

@app.route("/verify")
def verify_page():
```

```
    return render_template("verify.html")

@app.route("/check_otp", methods=["POST"])
def check_otp():
    user_otp = request.form['otp']
    if str(session['otp']) == user_otp:
        return "✅ OTP Verified Successfully!"
    else:
        return "❌ Invalid OTP. Try Again."

if __name__ == "__main__":
    app.run(debug=True)
```

Step 3: templates/send.html

```
<form action="/send_otp" method="post">
    <h2>Enter Phone Number</h2>
    <input type="text" name="phone" required>
    <button type="submit">Send OTP</button>
</form>
```

Step 4: templates/verify.html

```
<form action="/check_otp" method="post">
    <h2>Enter OTP</h2>
    <input type="text" name="otp" required>
    <button type="submit">Verify</button>
</form>
```

✅ **Output & Learning Achieved**

Concept	Learned	Used in Project
random module	Yes	Used for OTP generation
OTP generation	Yes	Implemented
Modularization	Yes	otp_utils.py used
Flask	Yes	Built small OTP app

🔑 What is app.secret_key ?

Flask uses **sessions** to store some temporary data **on the server** for each user.
But to keep this data **safe**, Flask needs a **secret key**.

So,

app.secret_key = "abc123"

means:

“Use this key to **securely lock** the session data.”

✅ Why needed?

- To **protect session data**
- To **avoid hacking / tampering**
- Required **before using session** object

⚠️ Note:

Never keep real projects secret key as simple as "abc123".
Use a strong random key like:

app.secret_key = "f3489f7adks9s8dfnsd38"

👜 What is session?

Session is like a **bag** where the server temporarily stores data for a user **until the browser closes or logout happens**.

Example:
We stored OTP in session:

session['otp'] = otp

This means:

Store OTP for this user so that we can check it later.

Later we verify:

if session['otp'] == user_entered_otp:

This means:

Compare the OTP the server stored with the OTP entered by the user.

🎯 **Simple Example**

```
from flask import Flask, session


app = Flask(__name__)

app.secret_key = "mysecret"


@app.route("/")

def store_data():

    session['username'] = "Jani"

    return "Data stored in session!"


@app.route("/get")

def get_data():

    return "Welcome " + session['username']
```

What happens here?

- When user visits /, the name **"Jani"** is stored in session.
- When user visits /get, Flask **remembers** the stored name and displays it.

✅ **Summary (Very Simple Words)**

Term	Meaning
secret_key	A **lock** used to secure session data

Term	Meaning
session	A **temporary storage bag** for storing user-related data